

The slide features a dark blue background with several thin, teal-colored lines. On the left side, there are three vertical lines that bend at different heights. At the bottom, there are several horizontal and diagonal lines that create a sense of depth and structure.

Dictionaries in Python

INTRODUCTION TO PYTHON DICTIONARIES

What is a Dictionary in Python?

- A dictionary is a collection of key-value pairs.
- Unordered, changeable, and does not allow duplicates.
- Syntax: {key: value, key: value, ...}

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
```

Creating a Dictionary

- Using {} to create a dictionary.
- Using dict() constructor.

```
my_dict = {"brand": "Ford", "model": "Mustang", "year": 1964}  
new_dict = dict(name="John", age=36, country="Norway")
```

Accessing Dictionary Elements

- Accessing values using keys.
- Using get() method.

```
print(my_dict["model"]) # Output: Mustang  
print(my_dict.get("year")) # Output: 1964
```

Dictionary Methods

ADDING OR CHANGING ITEMS

- Adding new key-value pairs.
- Modifying existing values.

```
my_dict["color"] = "red"  
my_dict["year"] = 2020
```

REMOVING ITEMS FROM A DICTIONARY

- pop(), popitem(), del, and clear() methods.

```
my_dict.pop("model") # Removes item with key 'model'  
my_dict.popitem() # Removes the last inserted key-value pair  
del my_dict["year"] # Deletes 'year' key  
my_dict.clear() # Clears the dictionary
```

Dictionary Methods

- `keys()`: Returns all keys.
- `values()`: Returns all values.
- `items()`: Returns key-value pairs.

```
keys = my_dict.keys()  
values = my_dict.values()  
items = my_dict.items()
```

copy() Method

- The copy() method creates a **shallow copy** of the dictionary.
- A shallow copy means that it copies the dictionary structure, but if the values themselves are mutable objects (like other dictionaries or lists), the references to those objects are copied, not the objects themselves. So, modifications to the nested mutable objects in the copy will also affect the original dictionary.

```
original = {"name": "Alice", "details": {"age": 25, "city": "New York"}}
copy_dict = original.copy()

copy_dict["details"]["age"] = 30

print(original["details"]["age"]) # Output: 30
```

dict() Constructor

- Using dict() also creates a **shallow copy** of the dictionary.
- Internally, dict() re-creates a new dictionary using the key-value pairs of the original dictionary.
- The behavior is almost identical to copy() in terms of copying the dictionary structure and references to mutable objects.

```
original = {"name": "Alice", "details": {"age": 25, "city": "New York"}}
dict_copy = dict(original)

dict_copy["details"]["city"] = "San Francisco"

print(original["details"]["city"]) # Output: "San Francisco"
```


Nested Dictionaries

- A dictionary within a dictionary.

```
family = {  
    "child1": {"name": "Emil", "year": 2004},  
    "child2": {"name": "Tobias", "year": 2007},  
}
```

Difference between List and tuple

	List	Tuple
Syntax	List is represented with square brackets []	Tuple is represented with parentheses ()
Mutable/Immutable	List is mutable object	Tuple is immutable object
Built-in methods	List supports many built-in methods such as insert, pop, remove, etc.	Tuple doesn't support as many built-in methods as list
Storage/memory Efficiency	List consumes a lot of memory/storage	Tuple consumes a lot less storage/memory
Time Efficiency	Creation of list and accessing the list elements is slower	Creation of tuple and accessing the tuple elements is faster

Comparision between all data structure

LIST	TUPLE	DICTIONARY	SET
Allows duplicate members	Allows duplicate members	No duplicate members	No duplicate members
Changeble	Not changeable	Changeable indexed	Cannot be changed, but can be added, non -indexed
Ordered	Ordered	Unordered	Unordered
Square bracket []	Round brackets ()	Curly brackets{ }	Curly brackets{ }



Thank You

HAPPY LEARNING!